

Software Engineering

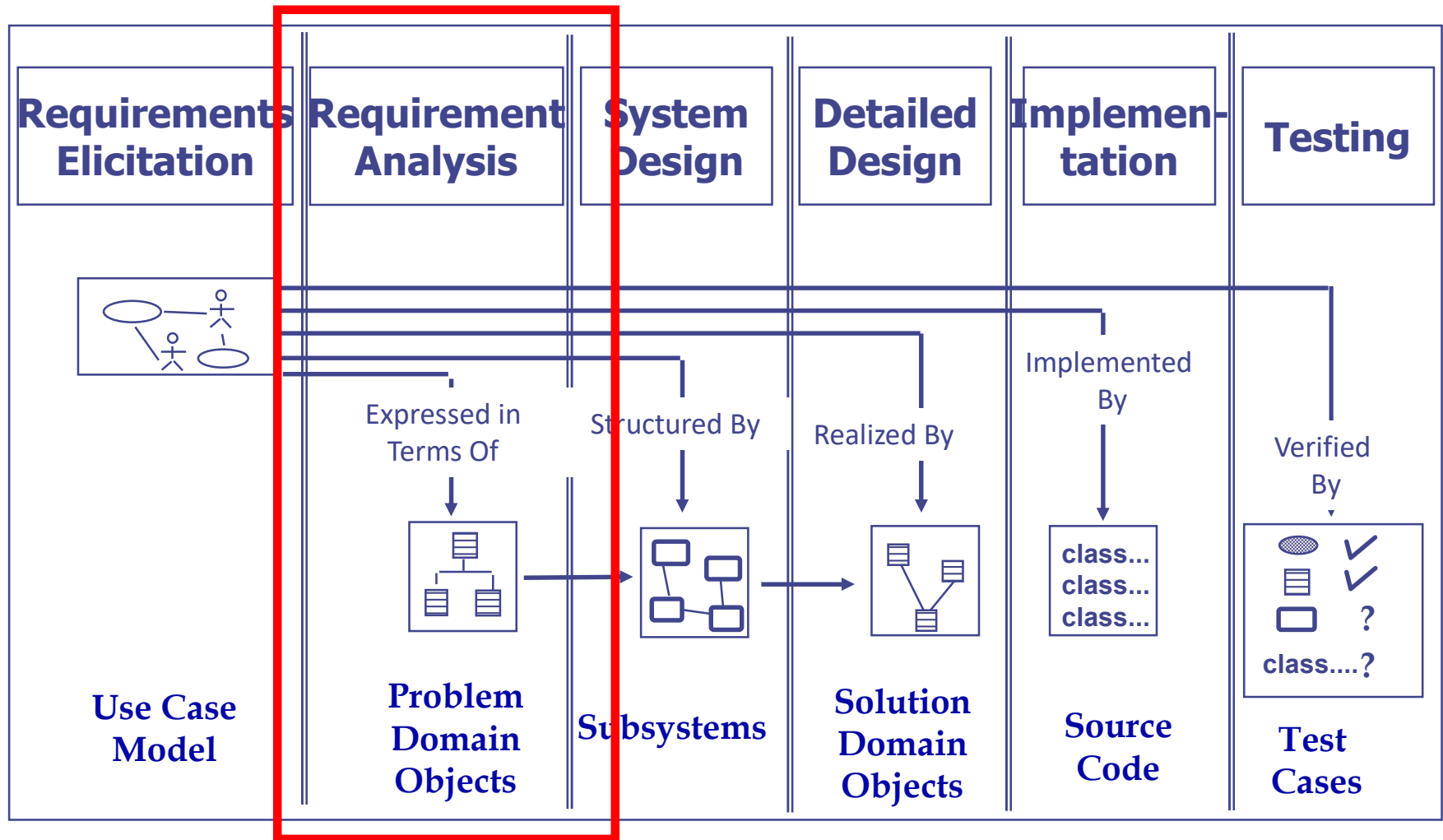
CE2006/CZ2006

Requirements Analysis: Conceptual Models

Discussion Topics

- Analysis Modeling
- Class Diagram
- Reading
 - Bruegge Chap 2.3 Modelling Concepts
Chap 2.4.2 Class Diagrams
 - Fowler Chap 3 Class Diagrams
Chap 6 Object Diagrams

Software Development Lifecycle Activities



Requirements Elicitation versus Requirements Analysis

- **Requirements Elicitation:**
 - **Purpose:** finding out what **customers** want
 - **Output:** a description of the system in terms of actors and use cases
- **Requirements Analysis:**
 - **Purpose:** produce a system model that is correct, complete, consistent, unambiguous based on use cases
 - **Output:** conceptual model (system structure)
+ dynamic model (system behavior)

Formalizing Requirements with Analysis Models

- Clarifies structural and dynamic aspects of system to be build
- Validates requirements
- Underpins solution modelling

Requirement Analysis Goals

- Conceptual model (structural aspect)
 - Analyze use cases to identify the **objects** and **roles** of objects involved in the system
- Dynamic model (dynamic aspect)
 - Determine how to **fulfill the processes** defined in the use cases and which **objects** **do these processes**

Conceptual Model

- **Objects:** Anything that has a state and exhibits behavior
 - e.g., Bus, Dog, Company
 - e.g., LoginForm, UserVerifier, UserList
- **Operations:** Procedures through which objects communicate amongst themselves
 - e.g.: Bus: Stop, Start. Dog: Bark, Growl. Company: Sell, Quote.
 - e.g., LoginForm: submit. UserVerifier: authenticate. UserList: SearchUser
- **Attributes:** Variables that hold state information
 - e.g. Bus: Seats, Colour. Dog: Name, Breed. Company: Name, Employees

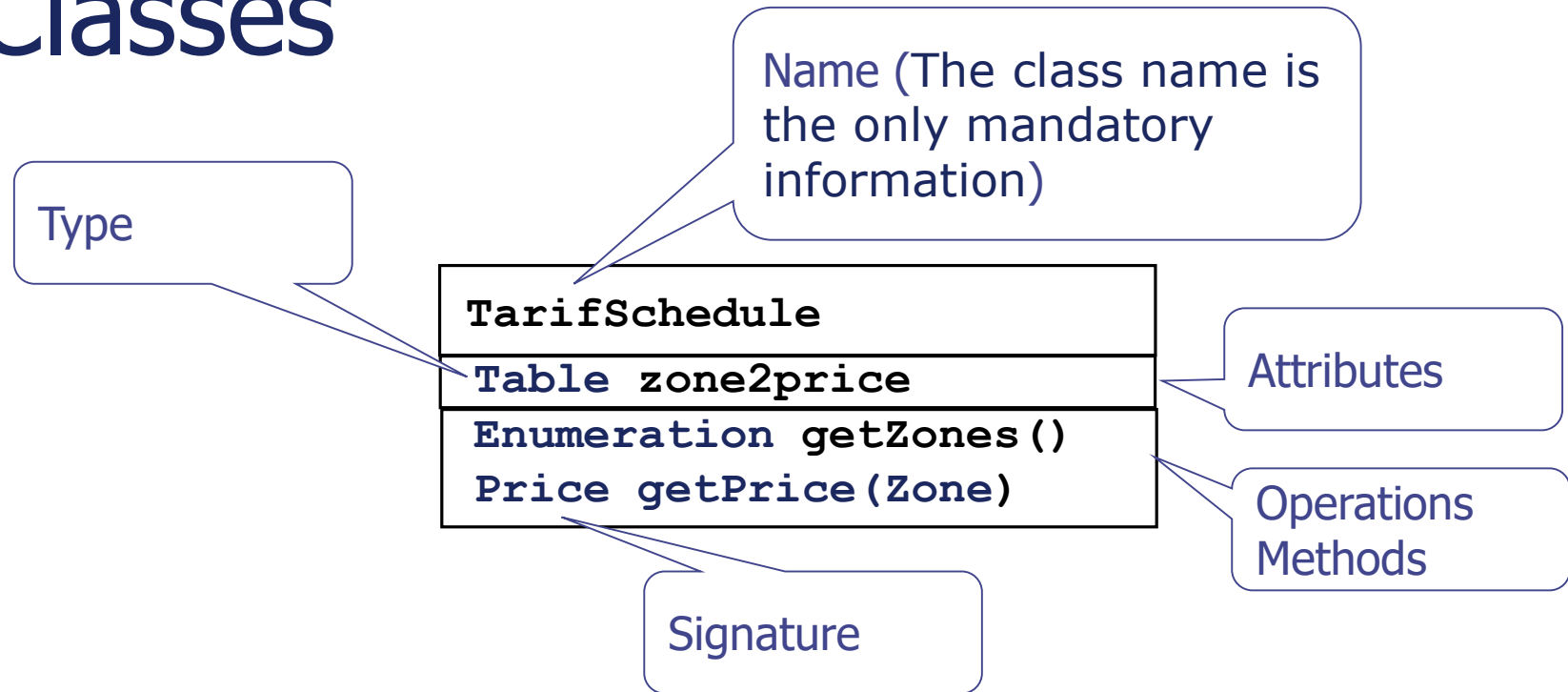
Conceptual Model in Class Diagram

- Document and visualize conceptual model in class diagram
 - Object $\Rightarrow$ Class
 - Roles (Attributes and operations) of object $\Rightarrow$ Attributes and methods of class

Class Diagrams

- Class diagrams represent the structure of the system
- The classes define the responsibilities for doing various activities
- Used during
 - requirements analysis: model application domain concepts
 - system design: model subsystems
 - object design: specify the detailed behavior and attributes of classes

Classes

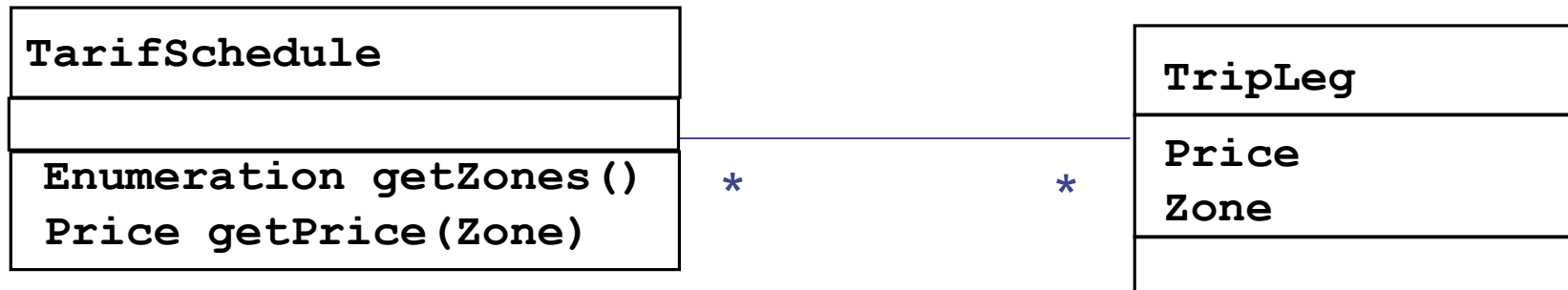


- A ***class*** represents a concept
- A class encapsulates states (***attributes***) and behaviour (***operations***)

Each attribute has a *type*

Each operation has a *signature*

Associations

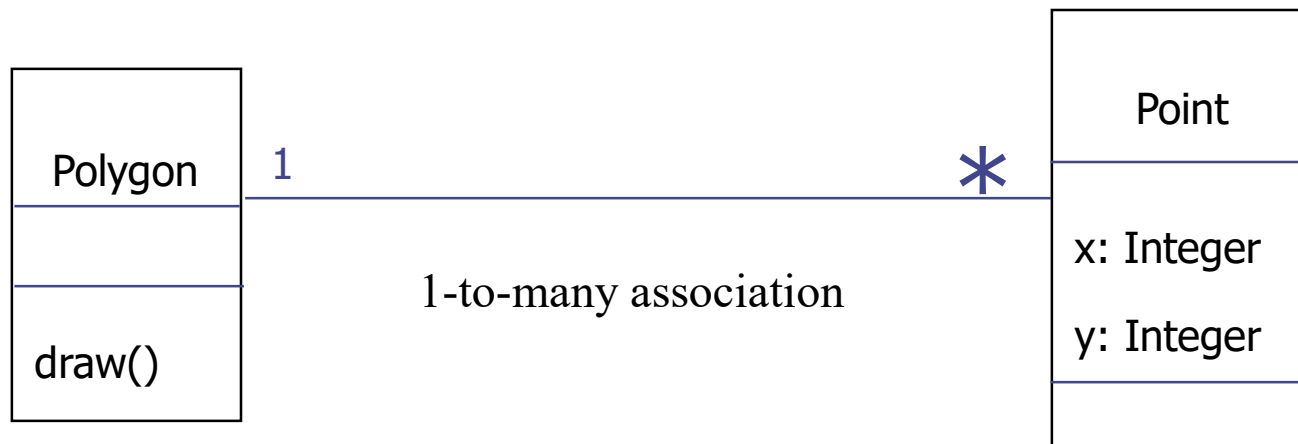
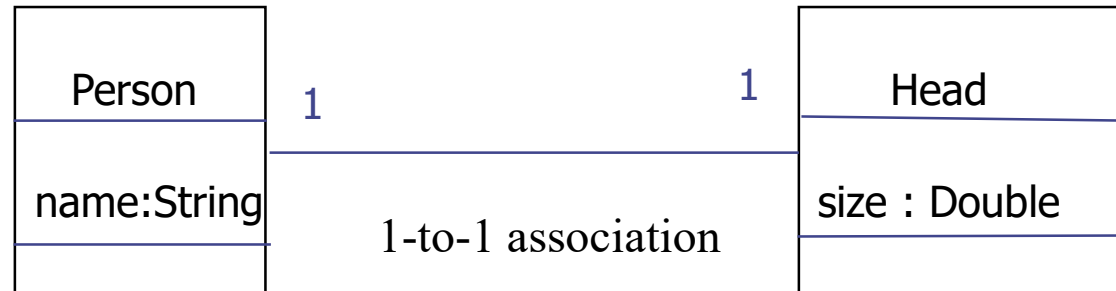


Associations denote relationships between classes

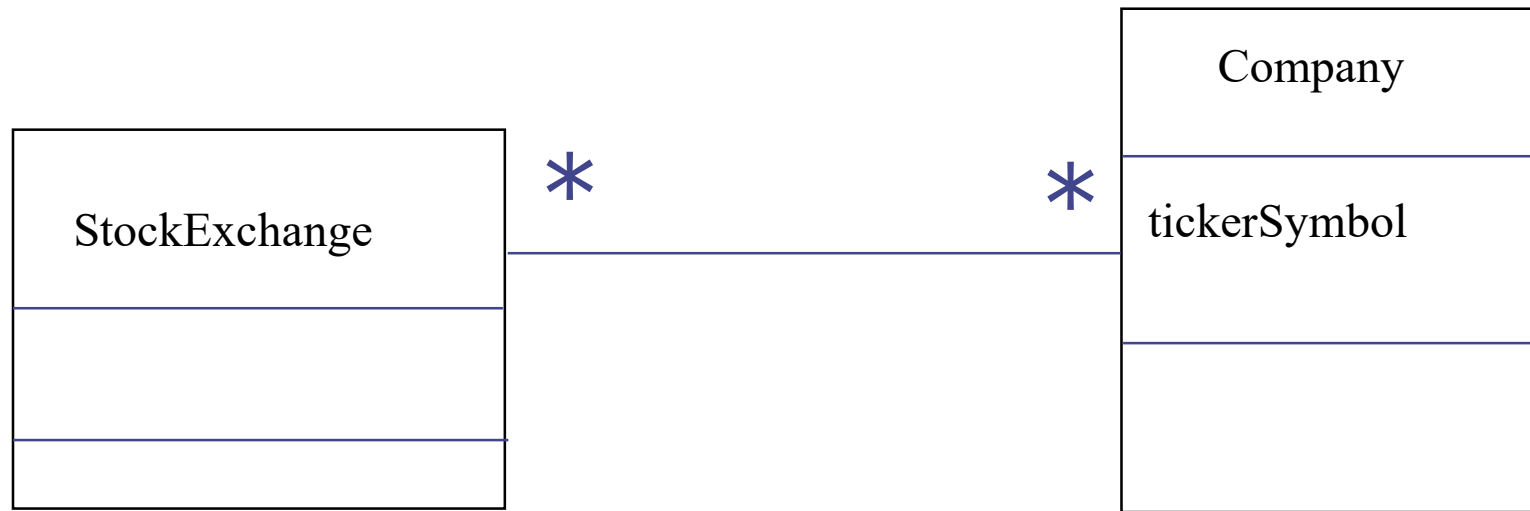
The multiplicity of an association end denotes how many objects the instance of a class can legitimately reference.

Multiplicity is a number (e.g. 3), a range of numbers (1..6) or a large unspecified number (many) (*)

1-to-1 and 1-to-many Associations



Many-to-Many Associations

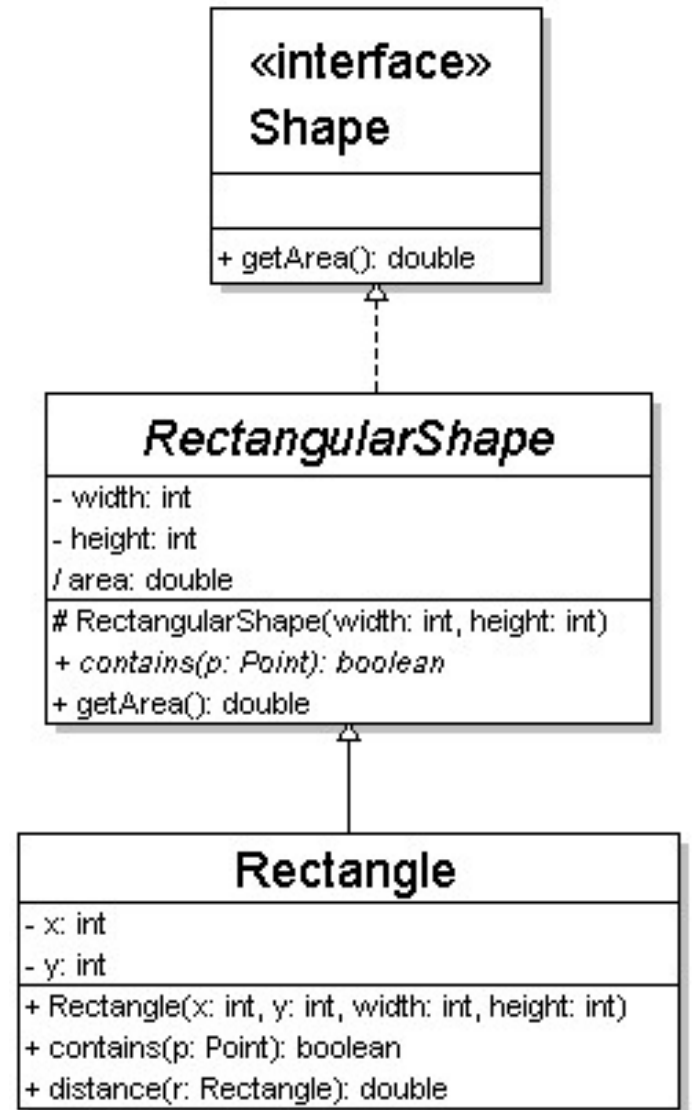


Class Relationships

- **generalization**: an inheritance relationship
 - inheritance between classes
 - interface implementation
- **association**: a usage relationship
 - dependency
 - aggregation
 - composition

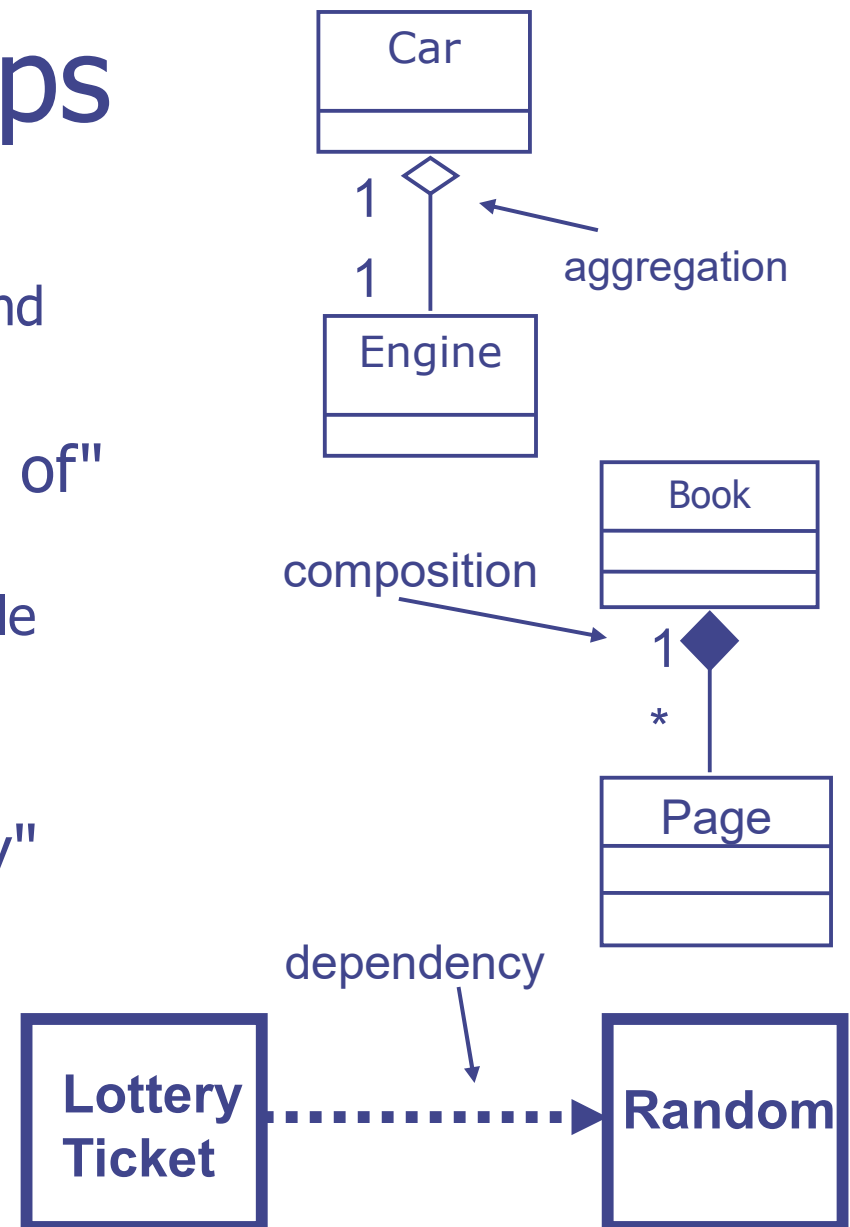
Class Relationships

- generalization (inheritance) relationships
 - hierarchies drawn top-down with arrows pointing upward to parent
 - line/arrow styles differ, based on whether parent is a(n):
 - class:
solid line, black arrow
 - abstract class:
solid line, white arrow
 - interface:
dashed line, white arrow
- we often don't draw trivial / obvious generalization relationships, such as drawing the Object class as a parent



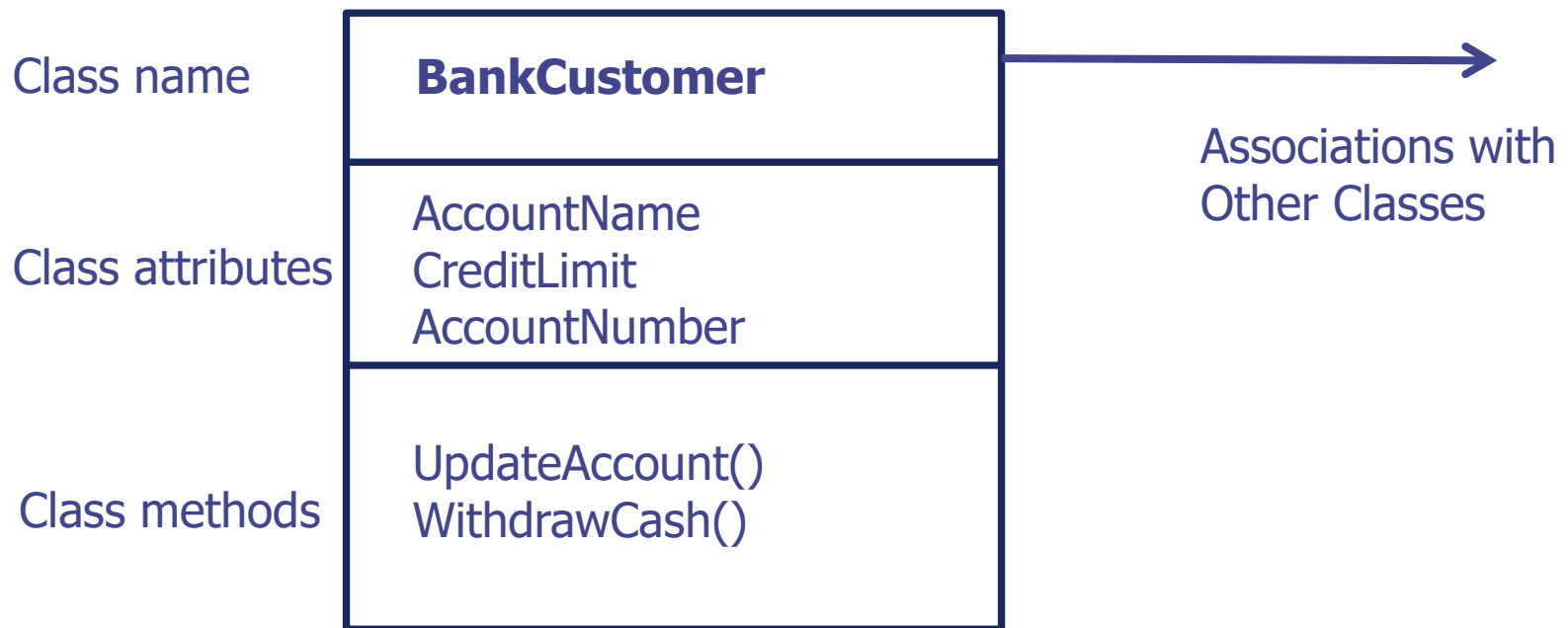
Class Relationships

- **aggregation:** "is part of"
 - symbolized by a clear white diamond
- **composition:** "is entirely made of"
 - stronger version of aggregation
 - the parts live and die with the whole
 - symbolized by a black diamond
- **dependency:** "uses temporarily"
 - symbolized by dotted line
 - often is an implementation detail, not an intrinsic part of that object's state



Requirement Analysis and Conceptual Modeling Activities

1. Identify the Classes Use case model → Class diagram
2. Find the attributes for each Class
3. Find the methods/operations for each Class
4. Find the associations between Classes



Identification of Initial Objects or Classes

Look for

1. Recurring nouns / concepts in the use cases
2. Real-world entities the system must track
3. Application (problem) domain terms in data dictionary

Review definitions and attributes with stakeholders

Stereotypes of Classes

 **Boundary Class <<boundary>>**
– interaction between Actor & System

 **Control Class <<control>>**
– logic to realize use case

 **Entity Class <<entity>>**
–information tracked by System

Types of Entity Classes

- Things remembered or data that persists
 - UserInfo, Event, Course, Book
- Organisational units
 - Company, TutorialGroup
- Structures
 - OrderList

Start with data dictionary

Example:

EnterSecureFacility Use Case Description

Identify recurring nouns, real-world entities,
domain terms – these may be Classes

Flow of Events:

1. User presses finger on FingerprintReader.
2. FingerprintReader scans fingerprint and sends image to the FacilityAccessSystem.
3. The FacilityAccessSystem validates fingerprint and unlocks EntryGate.
4. EntryGate sends a signal to the FacilityAccessSystem that it has opened.
5. The FacilityAccessSystem locks EntryGate, logs entry event and increments occupant count.

Example:

EnterSecureFacility Use Case Description

Identify recurring nouns, real-world entities, domain terms – these may be Classes

Flow of Events:

1. User presses finger on FingerprintReader.
2. FingerprintReader scans fingerprint and sends image to the FacilityAccessSystem.
3. The FacilityAccessSystem validates fingerprint and unlocks EntryGate.
4. EntryGate sends a signal to the FacilityAccessSystem that it has opened.
5. The FacilityAccessSystem locks EntryGate, logs entry event and increments occupant count.

Example:

EnterSecureFacility - Possible Classes

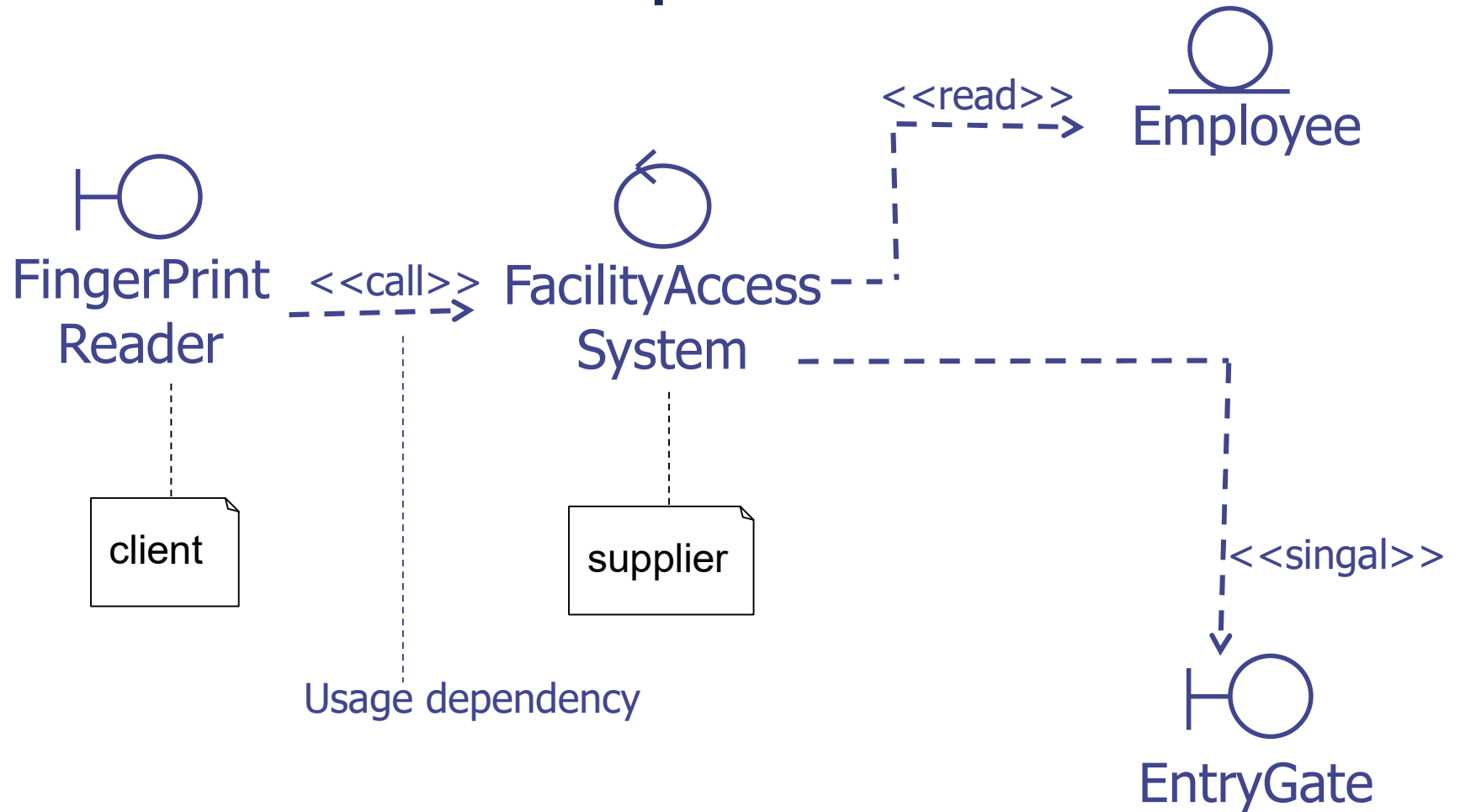
FingerprintReader

EntryGate

FacilityAccessSystem

Employee
(Fingerprint?)

Stereotyping to Indicate Class Responsibilities



Summary

- Analysis Modelling
- Class Diagram